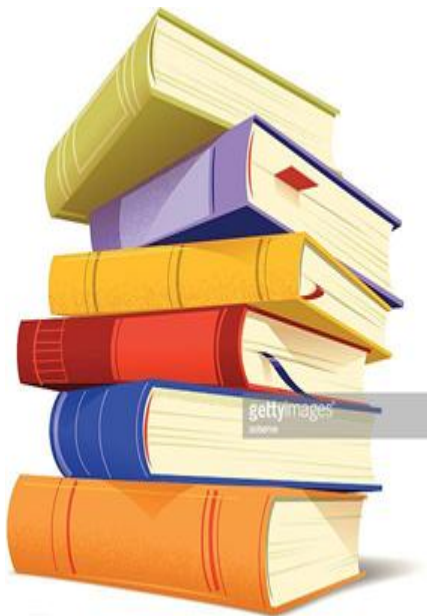


Lecture 2

Solving Problems by Searching (Part II)



Solving Problems by Searching (Part I)



- Informed (Heuristic) Search Strategies
- Heuristic Functions

A close-up photograph of a person's hand holding a round, silver-colored compass. The hand is wearing a dark jacket. The compass face is black with white markings for degrees and cardinal directions (N, E, S, W). The background is a blurred, snowy forest scene with snow-covered branches.

Informed Search

Introduction

Informed Search

AI search problems typically have a very large search space. We would like to improve efficiency by **expanding as few nodes as possible**.

Idea: The agent can use **additional information** in the form of “hints” about what promising states are to explore first. These hints are derived from

- **information** the agent has (e.g., a map with the goal location marked) or
- **percepts** coming from a sensor (e.g., a GPS sensor and coordinates of the goal).

Method: The agent uses a **heuristic function $h(n)$**

- to rank nodes in the frontier based on the additional information, and
- to select the most promising node in the frontier for expansion using the **best-first search** strategy.

Discussed algorithms:

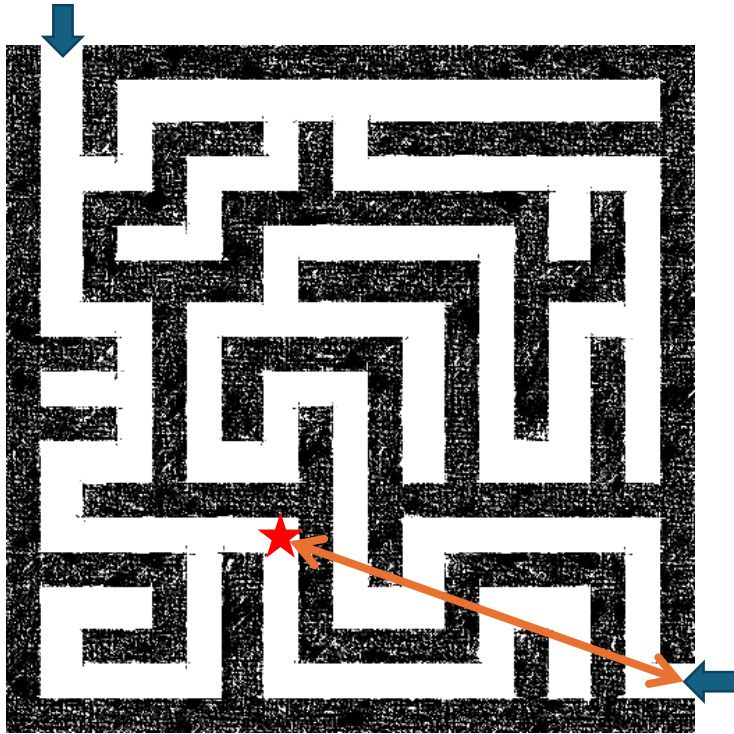
- Greedy best-first search
- A* search

Heuristic Function

- **Heuristic function** $h(n)$ estimates the cost of reaching a node representing the goal state from the currently considered node n .
- Examples:

Euclidean distance

Start state

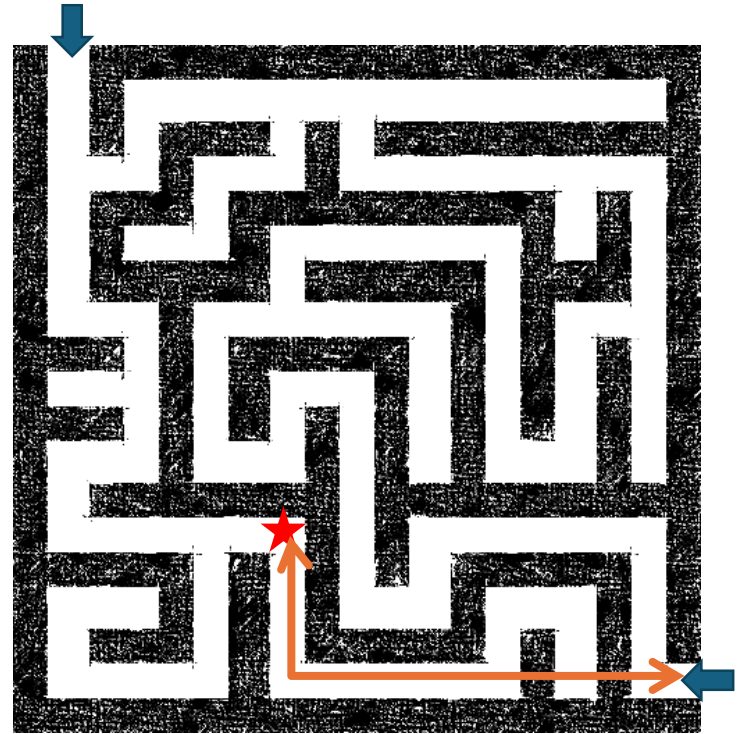


★ State for currently considered node

Goal state

Manhattan distance

Start state



Goal state

Heuristic for the Romania Problem

Use the map for hints: Estimate the driving distance from Arad to Bucharest using a straight-line distance on the map.

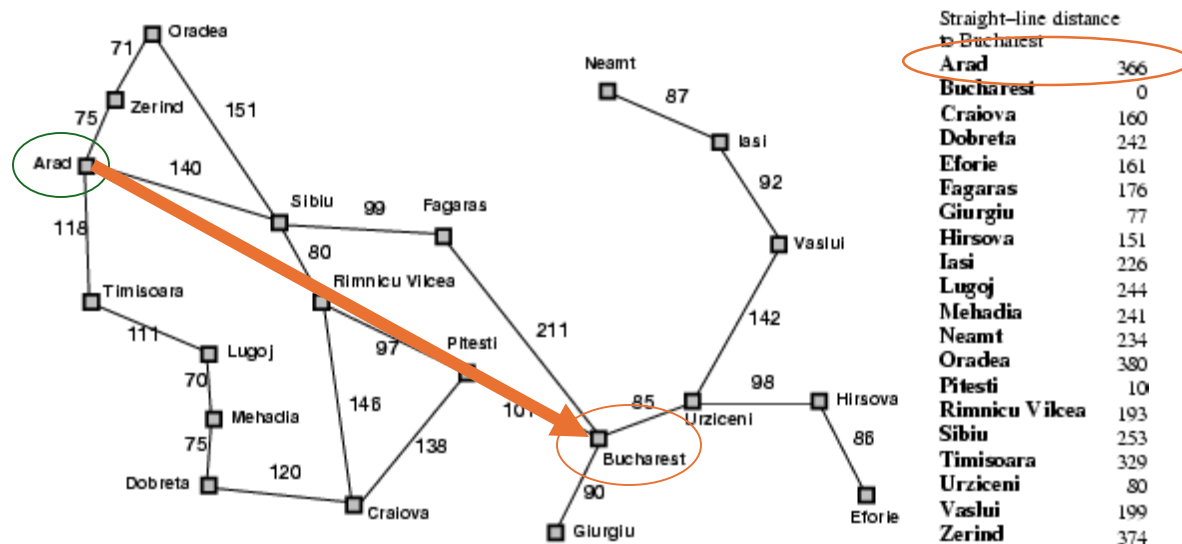


Straight-line distance to Bucharest	$h(n)$
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

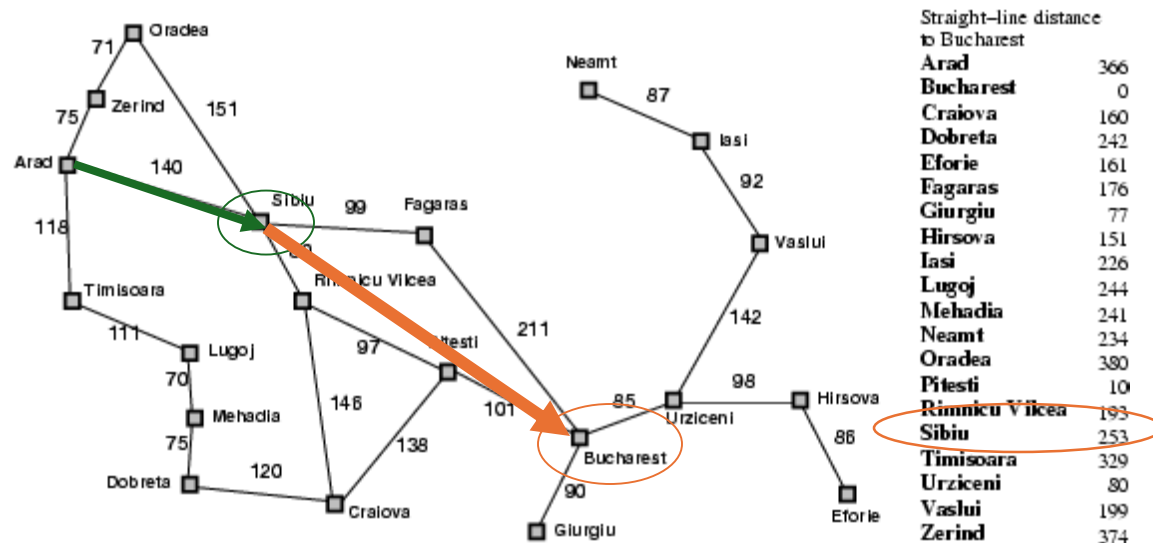
Greedy Best-First Search Example

Expansion rule: Expand the node that has the lowest value of the heuristic function $h(n)$

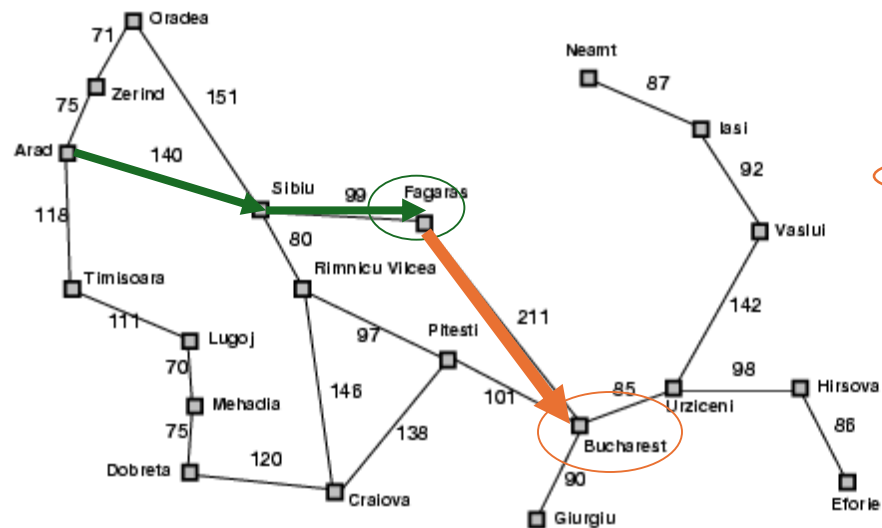
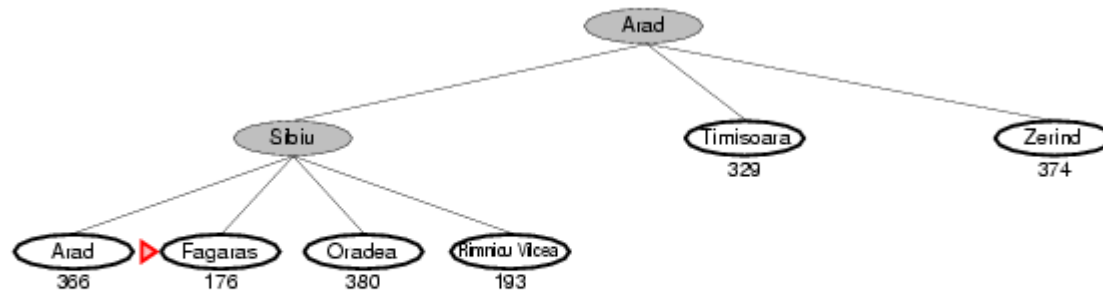
$$h(n) = \text{Arad} = 366$$



Greedy Best-First Search Example



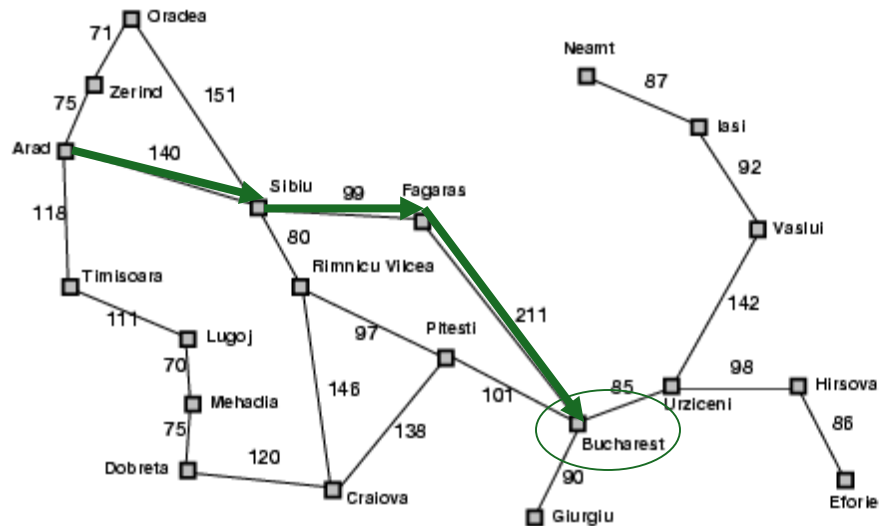
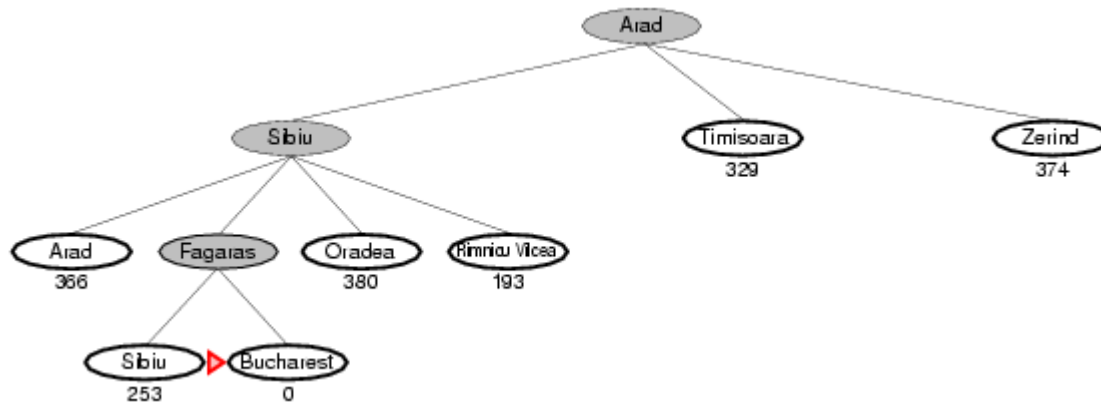
Greedy Best-First Search Example



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-First Search Example



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Total:

$$140 + 99 + 211 = 450 \text{ miles}$$

Properties of Greedy Best-First Search

- **Complete?**

Yes – Best-first search is complete in finite spaces.

- **Optimal?**

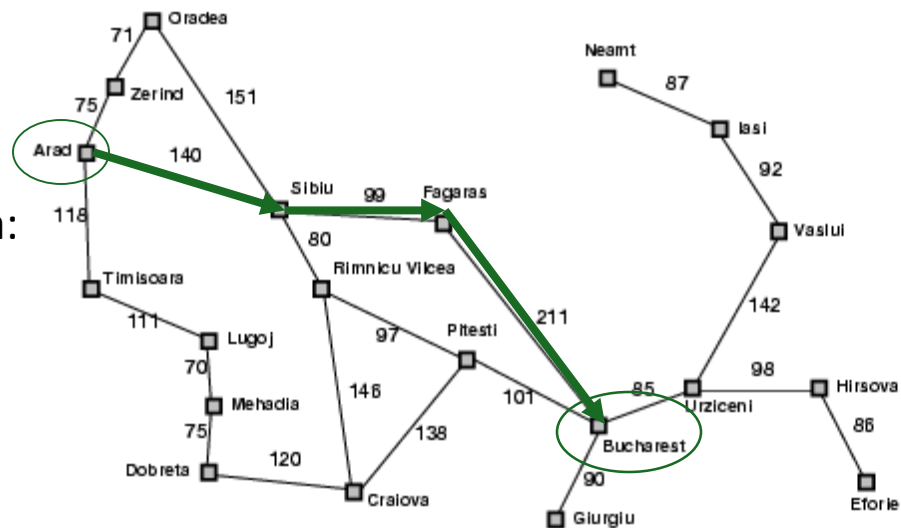
No

Total:

$$140 + 99 + 211 = 450 \text{ miles}$$

Alternative through Rimnicu Vilcea:

$$140 + 80 + 97 + 101 = 418 \text{ miles}$$



Implementation of Greedy Best-First search

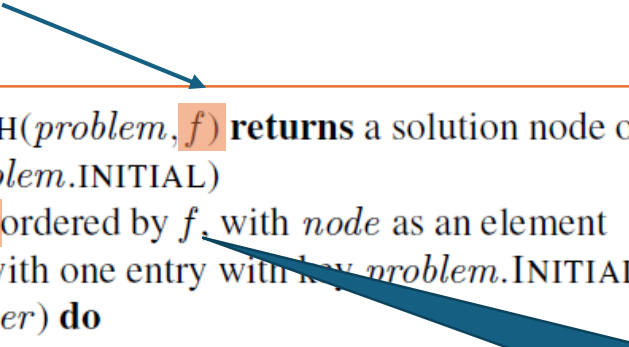
Best-First
Search



Expand the frontier
using
 $f(n) = h(n)$

Implementation of Greedy Best-First Search

Heuristic $h(n)$ so we expand the node with the lowest estimated cost



```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure  
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)  
  frontier  $\leftarrow$  a priority queue ordered by f, with node as an element  
  reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then  
        reached[s]  $\leftarrow$  child  
        add child to frontier  
  return failure
```

The order for expanding the frontier is determined by $f(n)$

See BFS for function EXPAND.

Properties of Greedy Best-First Search

- **Complete?**

Yes – Best-first search is complete in finite spaces.

- **Optimal?**

No

d : depth of the optimal solution
 m : max. depth of tree
 b : maximum branching factor

- **Time?**

Worst case: $O(b^m) \Leftrightarrow$ like DFS

Best case: $O(bm)$ – If $h(n)$ is 100% accurate we only expand a single path.

- **Space?**

Same as time complexity.

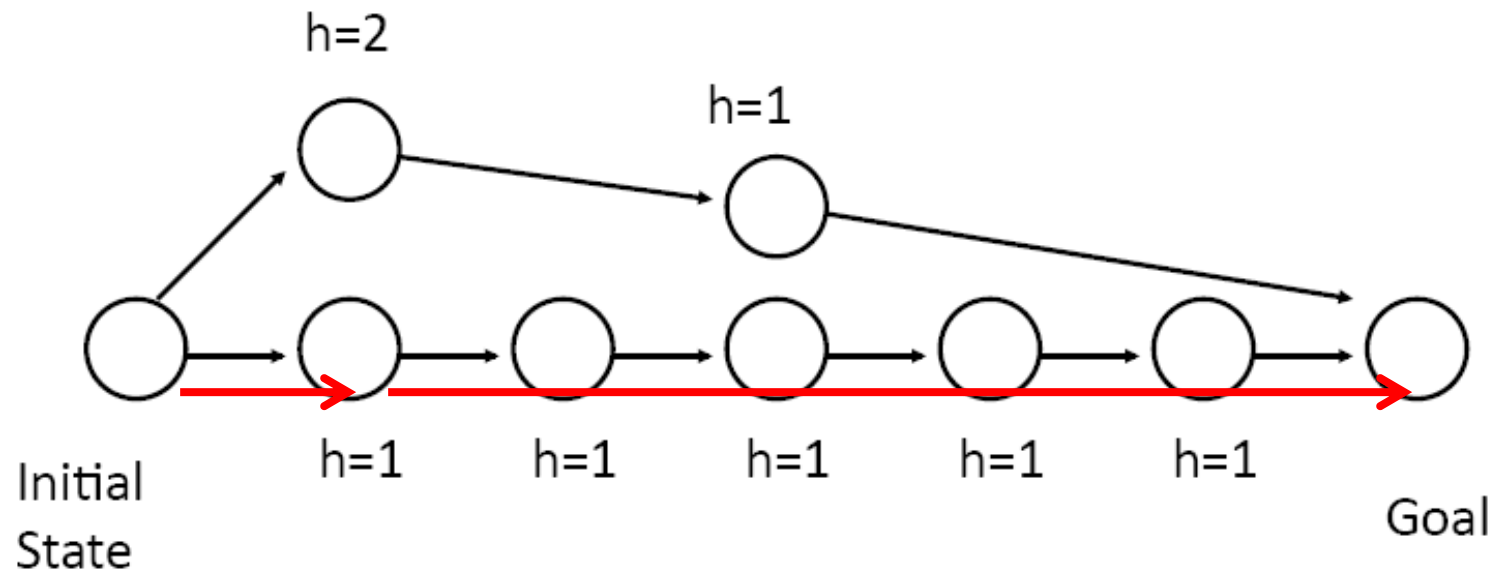


Informed Search

A* Search

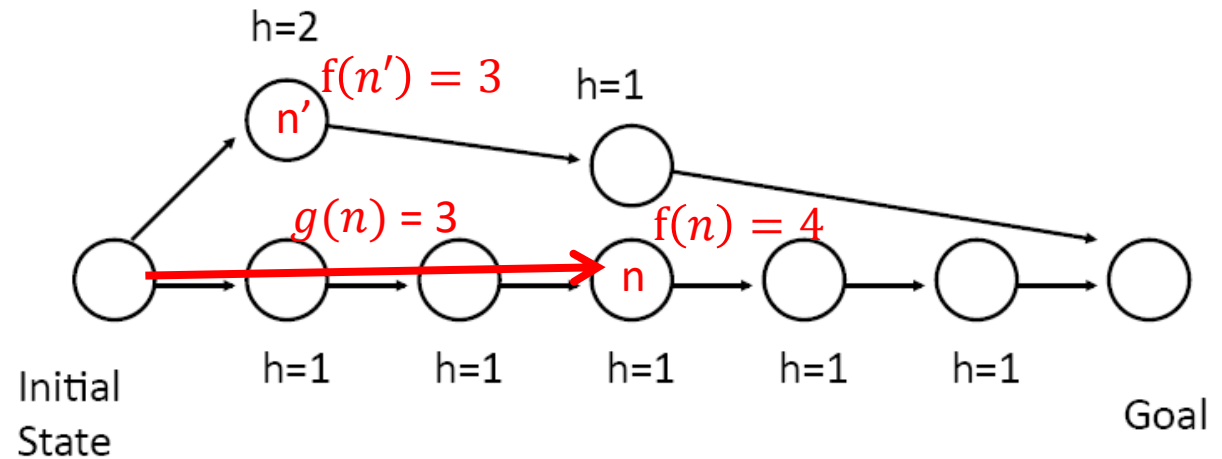
The Optimality Problem of Greedy Best-First search

Greedy best-first search only considers the estimated cost to the goal.



$h = 1$ is better than $h = 2$.
Greedy best-first will go this way
and never reconsider!

A* Search



- **Idea:** Take the cost of the path to n called $g(n)$ into account to avoid expanding paths that are already very expensive.
- The evaluation function $f(n)$ is the estimated total cost of the path through node n to the goal:

$$f(n) = g(n) + h(n)$$

$g(n)$: cost so far to reach n (path cost)

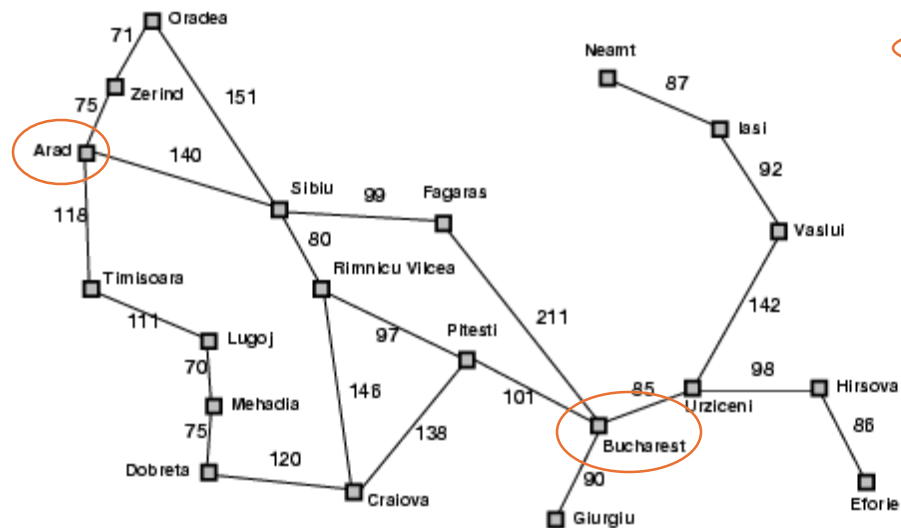
$h(n)$: estimated cost from n to goal (heuristic)

- The agent in the example above will stop at n with $f(n) = 3 + 1 = 4$ and chose the path up with a better $f(n') = 1 + 2 = 3$.

Note: For greedy best-first search we just used $f(n) = h(n)$.

A* Search Example

Expansion rule: $f(n) = g(n) + h(n) =$ Arad
 Expand the node with
 the smallest $f(n)$



$h(n)$

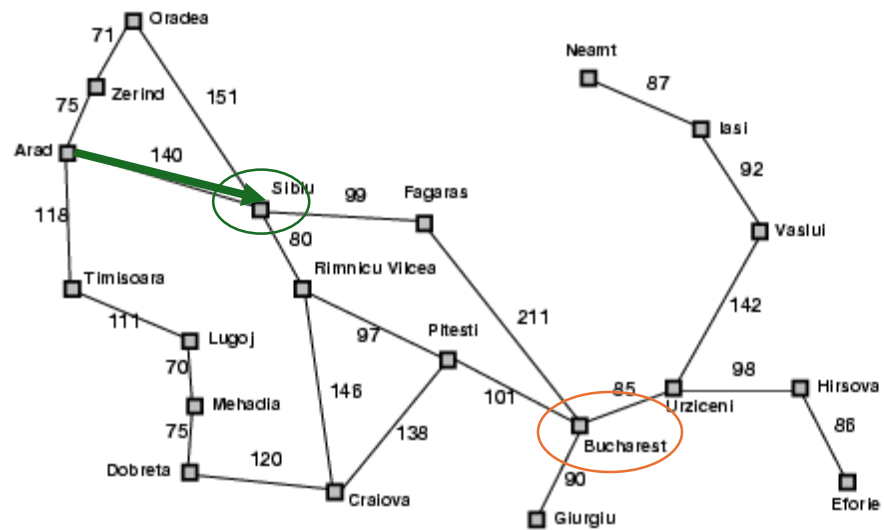
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example



$$f(n) = g(n) + h(n)$$

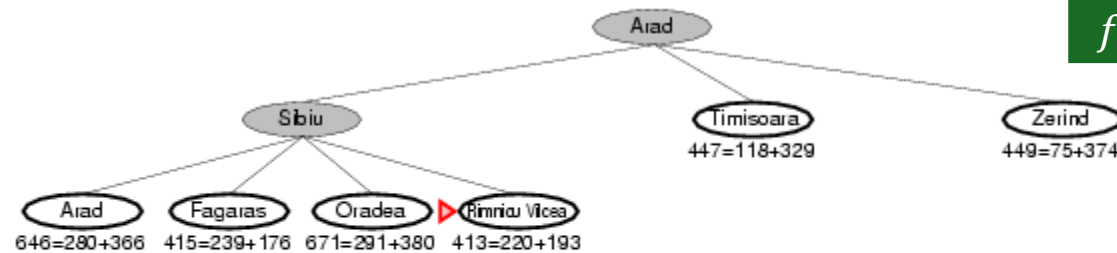


$h(n)$

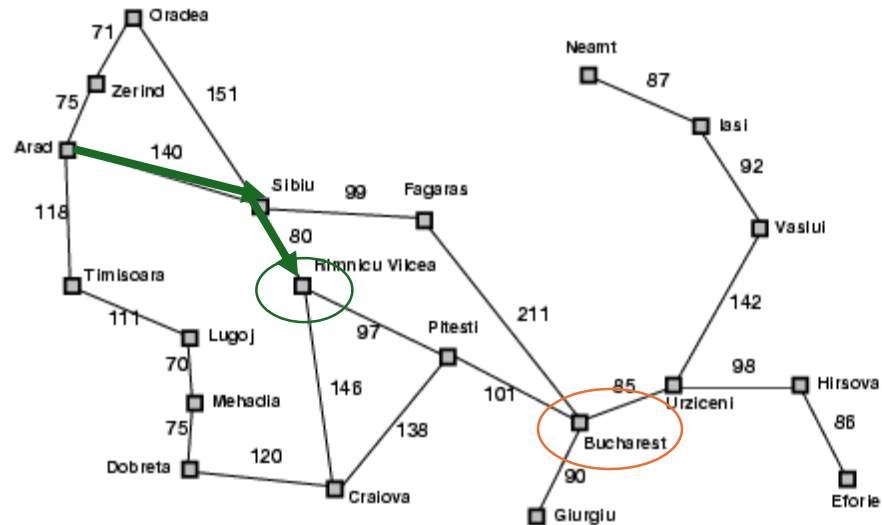
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example



$$f(n) = g(n) + h(n)$$



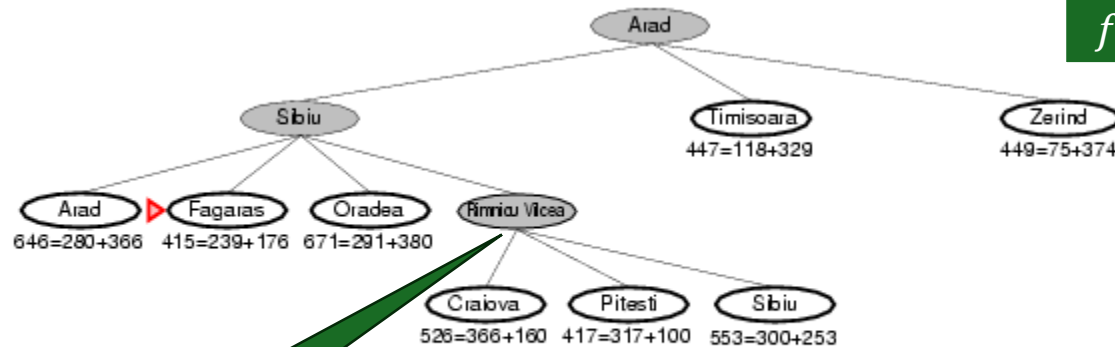
$h(n)$

Straight-line distance to Bucharest

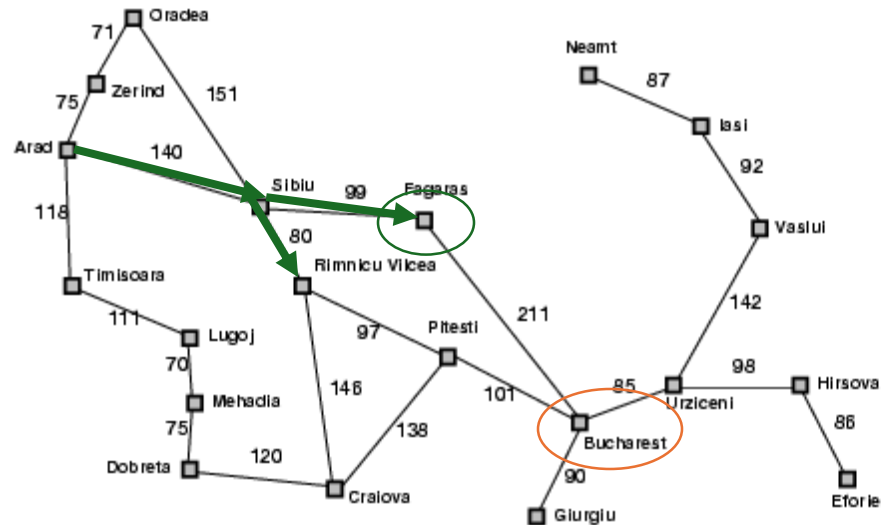
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example

$$f(n) = g(n) + h(n)$$



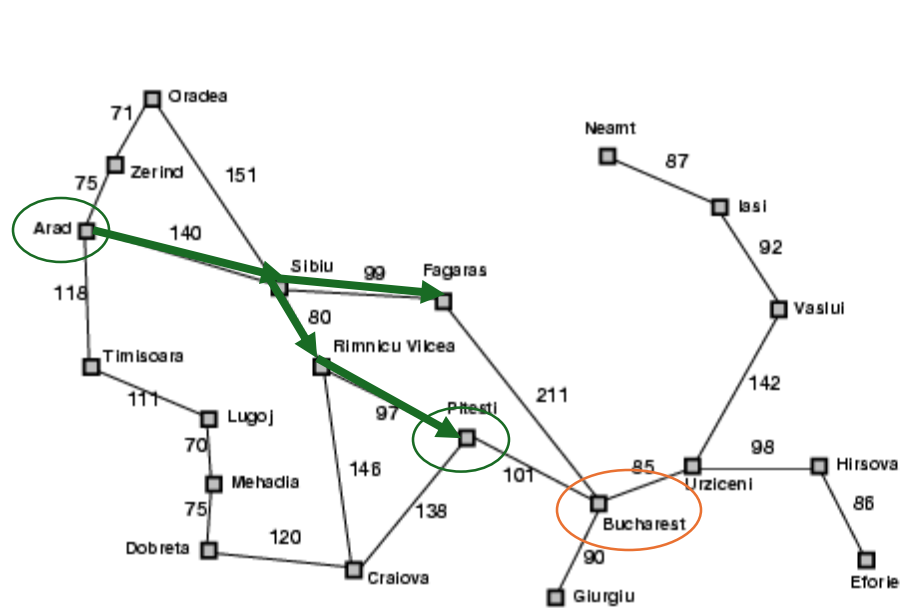
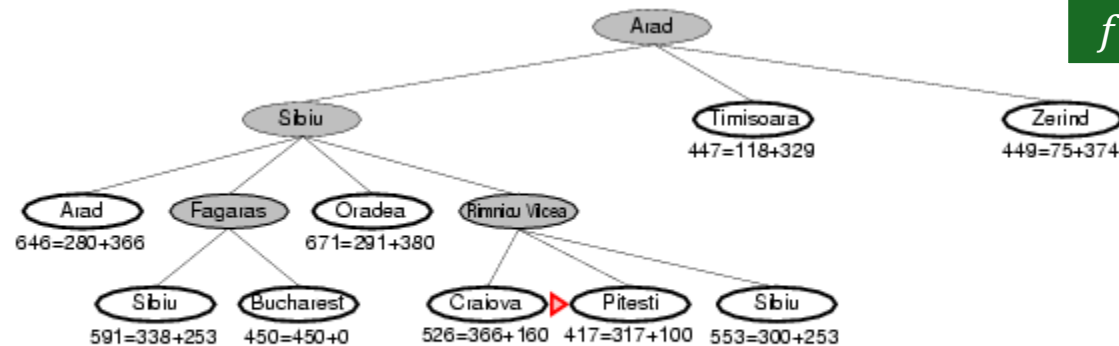
Reconsiders Rimnicu Vilcea because Fagaras may have a shorter total cost to Bucharest.



$h(n)$

Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example

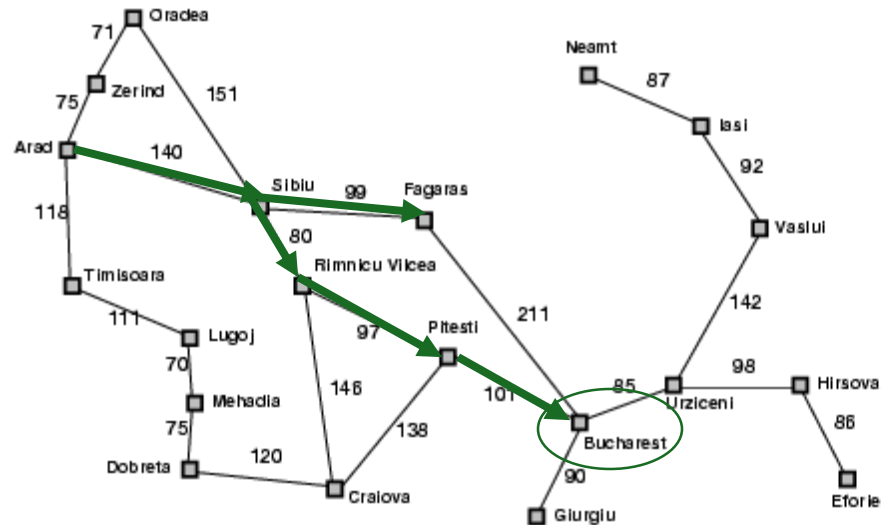
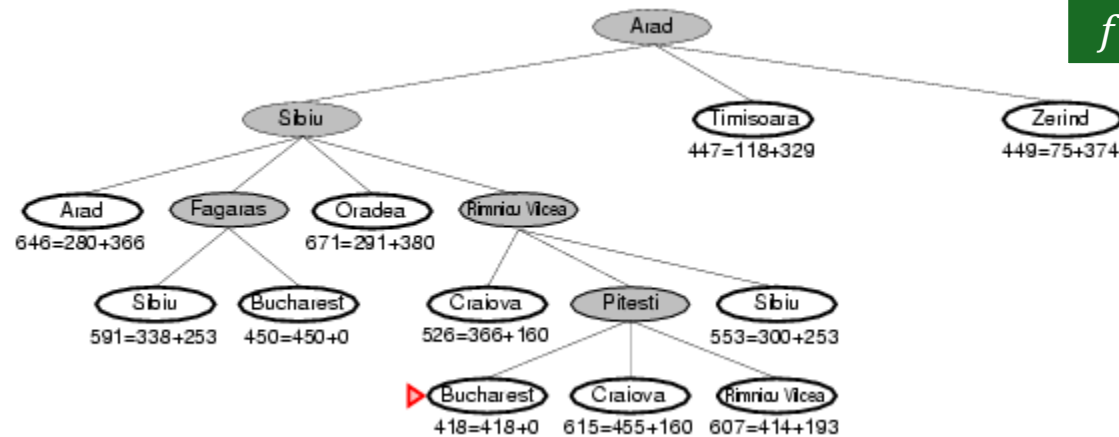


$h(n)$

Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example



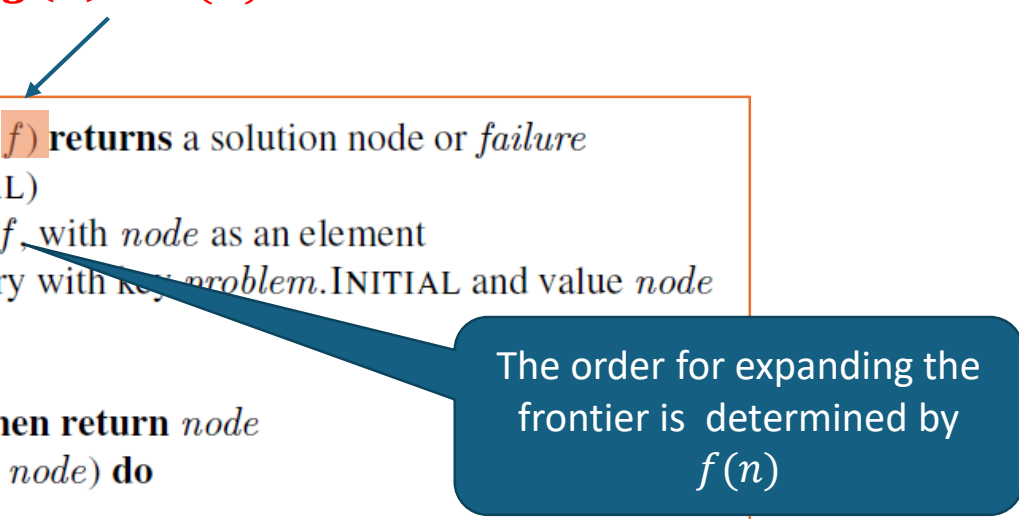
$h(n)$

Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Implementation of A* Search

Path cost to n + heuristic from n to goal = estimate of the total cost
 $g(n) + h(n)$



```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)
  frontier  $\leftarrow$  a priority queue ordered by f, with node as an element
  reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s  $\leftarrow$  child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s]  $\leftarrow$  child
        add child to frontier
  return failure
```

The order for expanding the frontier is determined by $f(n)$

See BFS for function EXPAND.

Optimality: Admissible Heuristics

Definition: A heuristic h is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n .

I.e., an admissible heuristic is a **lower bound** and never overestimates the true cost to reach the goal.

Example: Straight line distance never overestimates the actual road distance.

Theorem: If h is admissible, A^* is optimal.

Guarantees of A* Search

A* is **optimally efficient**

No other tree-based search algorithm that employs the same heuristic can expand fewer nodes and still guarantee the optimal solution.

Proof: Any algorithm that does not expand all nodes with $f(n) < C^*$ (the lowest cost of going to a goal node) cannot be optimal. It risks missing the optimal solution.

Properties of A*Search

- **Complete?**

Yes

- **Optimal?**

Yes

- **Time?**

Number of nodes for which $f(n) \leq C^*$ in the worst case $O(b^d)$ like BFS.

- **Space?**

Same as time complexity. This is often too high unless a very good heuristic is known.

A close-up photograph of a person's hand holding a round, silver compass. The hand is wearing a dark jacket. The compass face is black with white markings for degrees and cardinal directions (N, E, S, W). The background is a blurred, snowy forest with snow-covered branches.

Informed Search Designing Heuristics

Designing Heuristic Functions

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Example heuristics for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_1(start) = 8$

- $h_2(n)$ = total Manhattan distance (number of squares from the desired location of each tile)
- $h_2(start) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$

Are h_1 and h_2 admissible?

1 needs to move 3 positions

Heuristics from Relaxed Problems

- A problem with fewer restrictions on the actions is called a relaxed problem.
- **The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem. I.e., the true cost is never smaller.**
- What relaxation is used by h_1 and h_2 ?
 - h_1 : If the rules of the 8-puzzle are relaxed so that a tile can move **anywhere**, then $h_1(n)$ gives the shortest solution.
 - h_2 : If the rules are relaxed so that a tile can move to **any adjacent square**, then $h_2(n)$ gives the shortest solution.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

$$h_1(start) = 8$$

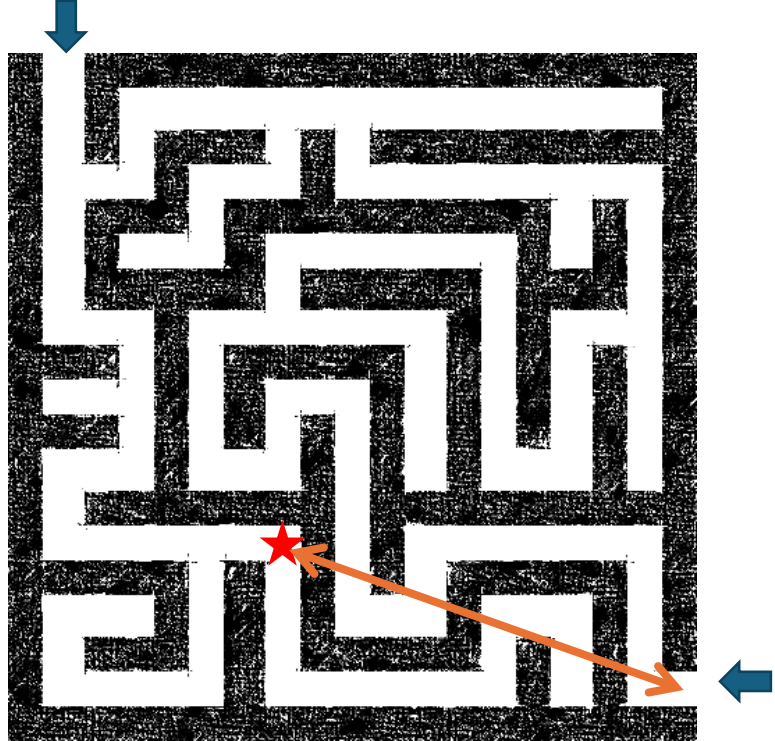
$$\begin{aligned} h_2(start) &= 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 \\ &= 18 \end{aligned}$$

Heuristics from Relaxed Problems

What relaxations are used in these two cases?

Euclidean distance

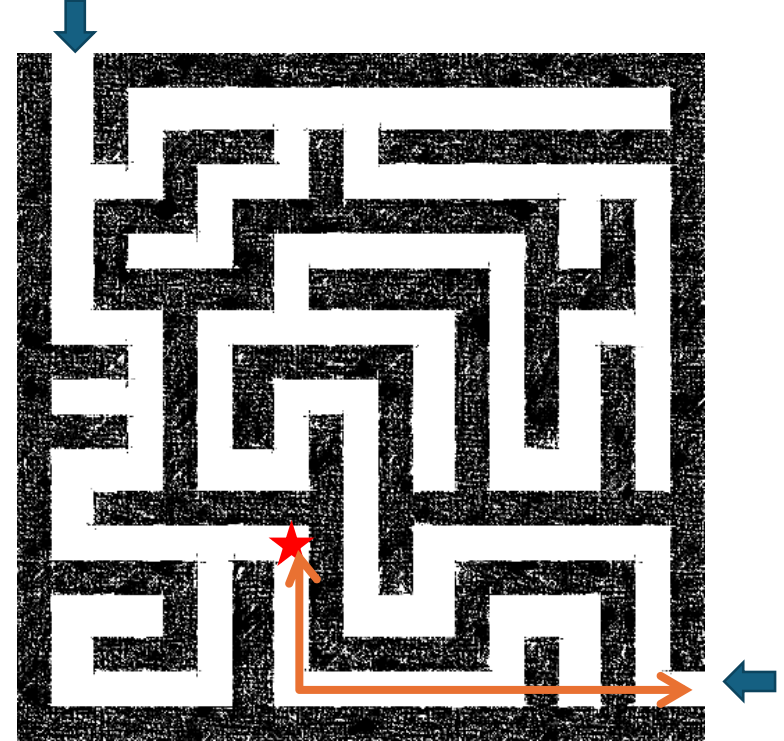
Start state



Goal state

Manhattan distance

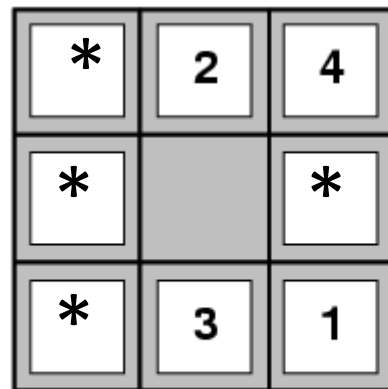
Start state



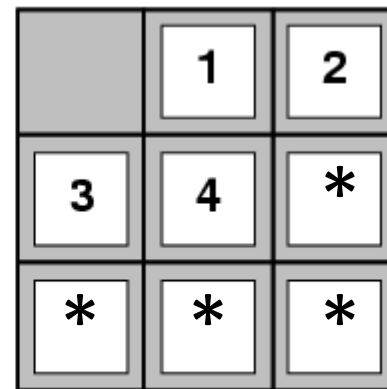
Goal state

Heuristics from Subproblems

- Let $h_3(n)$ be the cost of getting a subset of tiles (say, 1,2,3,4) into their correct positions. The final order of the * tiles does not matter.
- Solutions for subproblems are an admissible heuristic.
- Calculation:
 - Very small subproblems are often easy to solve.
 - We can precompute and save the exact solution cost for every or many possible subproblem instances – ***pattern database***.



Start State



Goal State

Dominance: What Heuristic is Better?

Definition: If h_1 and h_2 are both admissible heuristics and $h_2(n) \geq h_1(n)$ for all n , then h_2 dominates h_1

Is h_1 or h_2 better for A* search?

- A* search expands every node with $f(n) < C^* \Leftrightarrow h(n) < C^* - g(n)$
- h_2 is never smaller than h_1 . A* search with h_2 will expand less nodes and is therefore better.
- C^* = Cost of the optimal path

Vì sao h_2 làm A^* expand ít node hơn?

- Vì h_2 lớn hơn $\rightarrow f(n)$ lớn hơn:
$$f_2(n) = g(n) + h_2(n) \geq g(n) + h_1(n) = f_1(n)$$
 - Hệ quả:
 - Nhiều node có: $f_1(n) < C^*$ nhưng $f_2(n) \geq C^*$
 - Những node này:
 - Với $h_1 \rightarrow$ bị expand !
 - Với $h_2 \rightarrow$ bị loại ✕
- $\rightarrow$ Giảm số node cần duyệt**

Combining Heuristics

- Suppose we have a collection of admissible heuristics $h_1, h_2, \dots, h_m$, but none of them dominates the others.
- Combining them is easy:

$$h(n) = \max\{h_1(n), h_2(n), \dots, h_m(n)\}$$

- That is, always pick for each node the heuristic that is closest to the real cost to the goal $h^*(n)$.

Example: Effect of Information in Search

Typical search costs for the 8-puzzle

- State space: $\frac{9!}{2} = 1,811,440$ states
- Problem with solution at depth $d = 12$
 - IDS = 3,644,035 nodes
 - $A^*(h_1) = 227$ nodes
 - $A^*(h_2) = 73$ nodes
- Solution at depth $d = 24$
 - IDS $\approx 54,000,000,000$ nodes
 - $A^*(h_1) = 39,135$ nodes
 - $A^*(h_2) = 1,641$ nodes

7	2	4
5		6
8	3	1

$h_1(n)$ = number of misplaced tiles
 $h_2(n)$ = total Manhattan distance

Contains many
redundant paths which
IDS cannot break!

Satisficing Search: Weighted A* Search

- Often it is sufficient to find a **“good enough” solution** if it can be found very quickly or with way less computational resources. I.e., **expanding fewer nodes**.
- We could use inadmissible heuristics in A* search (e.g., by multiplying $h(n)$ with a factor W) that sometimes overestimate the optimal cost to the goal slightly.
 1. It potentially reduces the number of expanded nodes significantly.
 2. **This will break the algorithm’s optimality guaranty!**

$$f(n) = g(n) + W \times h(n)$$

Weighted A* search:	$g(n) + W \times h(n)$	$(1 < W < \infty)$
----------------------------	------------------------	--------------------

The presented algorithms are special cases:

A* search:	$g(n) + h(n)$	$(W = 1)$
Uniform cost search/BFS:	$g(n)$	$(W = 0)$
Greedy best-first search:	$h(n)$	$(W = \infty)$



Summary Planning Agents

Summary: All Search Strategies

b: maximum branching factor of the search tree
 d: depth of the optimal solution
 m: maximum length of any path in the state space
 C*: cost of optimal solution

Algorithm	Complete?	Optimal?	Time complexity	Space complexity
BFS (Breadth-first search)	Yes	If all step costs are equal	$O(b^d)$	$O(b^d)$
Uniform-cost Search	Yes	Yes	Number of nodes with $g(n) \leq C^*$	
DFS	In finite spaces (cycles checking)	No	$O(b^m)$	$O(bm)$
IDS	Yes	If all step costs are equal	$O(b^d)$	$O(bd)$
Greedy best-first Search	In finite spaces (cycles checking)	No	Depends on heuristic Best case: $O(bd)$ Worst case: $O(b^m)$	
A* Search	Yes	Yes	Number of nodes with $g(n) + h(n) \leq C^*$ With a good heuristic	

Needs cycle checking
Cannot avoid redundant paths

Implementation as Best-First Search

- All discussed search strategies can be implemented using Best-first search.
- Best-first search expands always the **node with the minimum value of an evaluation function $f(n)$** .

Search Strategy	Evaluation function $f(n)$
BFS (Breadth-first search)	$g(n)$ (=uniform path cost)
Uniform-cost Search	$g(n)$ (=path cost)
DFS/IDS (see note below!)	$-g(n)$
Greedy Best-first Search	$h(n)$
(weighted) A* Search	$g(n) + W \times h(n)$

- **Important note:** Do not implement DFS/IDS using Best-first search! You will get poor space complexity from BFS and the disadvantages of DFS (not optimal and worse time complexity).